

Siddarth B

CS22B1099

 Project midsem

Document Details

Submission ID

trn:oid:::3618:129620374

Submission Date

Mar 2, 2026, 10:40 AM GMT+5:30

Download Date

Mar 2, 2026, 10:51 AM GMT+5:30

File Name

CS22B1099.pdf

File Size

399.2 KB

29 Pages

5,440 Words

31,183 Characters

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **34 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 3%  Publications
- 5%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 34 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 3% Publications
- 5% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... 2%
- 2** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%
- 3** Publication
Artiom Alhazov, Rudolf Freund, Sergiu Ivanov, Sergey Verlan. "Numerical networ... <1%
- 4** Publication
Gexiang Zhang, Mario J. Pérez-Jiménez, Marian Gheorghe. "Real-life Applications ... <1%
- 5** Internet
reintech.io <1%
- 6** Internet
grotrian.nsu.ru <1%
- 7** Publication
"Medical Image Computing and Computer Assisted Intervention – MICCAI 2024", ... <1%
- 8** Publication
Prajapati, Bhagirath Parshuram. "Knowledge Discovery and Data Mining in Agric... <1%
- 9** Publication
"Beyond Artificial Intelligence", Springer Science and Business Media LLC, 2013 <1%
- 10** Student papers
Indian Institute of Information Technology, Design and Manufacturing - Kanchee... <1%

11	Internet	idus.us.es	<1%
12	Internet	www.mdpi.com	<1%
13	Student papers	Indian Institute of Information Technology, Design and Manufacturing - Kanchee...	<1%
14	Student papers	University Politehnica of Bucharest on 2018-03-30	<1%
15	Internet	digitalarchive.boun.edu.tr	<1%
16	Internet	vitalflux.com	<1%
17	Internet	www.gurulk.com	<1%
18	Internet	zombiedoc.com	<1%
19	Publication	Gutierrez-Naranjo, M.A.. "On the degree of parallelism in membrane systems", Th...	<1%
20	Publication	Jinyu Yang, Hong Peng, Xiaohui Luo, Jun Wang. "Stochastic Numerical P Systems ...	<1%
21	Internet	www.research.unipd.it	<1%
22	Student papers	Indian Institute of Technology, Kharagpure on 2016-02-05	<1%
23	Student papers	University of South Africa on 2026-02-24	<1%
24	Student papers	University of Strathclyde on 2012-02-17	<1%

25	Internet	
link.altmetric.com		<1%
26	Internet	
webthesis.biblio.polito.it		<1%
27	Student papers	
Heriot-Watt University on 2021-04-22		<1%

ABSTRACT

This project introduces a new technique for storing Machine Learning algorithms inside a special type of Numerical P Systems called Numerical P Systems with Boolean Conditions (BNPS). BNPS use real-valued variables to calculate and use arithmetic production rules. A simulator for BNPS running on GPUs has been developed, supporting both single-membrane and multi-membrane configurations. This simulator performs all three phases of BNPS execution — evaluating conditions, computing production values, and redistributing outputs via repartition — either sequentially using Python or in parallel using a CUDA kernel on a GPU.

Linear regression and logistic regression were designed and coded through a multiple stage pipeline architecture using the BNPS framework. Linear regression uses Batch Gradient Descent as the training method with a Mean Squared Error loss function based on a synthetic dataset containing six samples. Logistic regression uses a piecewise-linear approximation of the sigmoid function to enable gradient-based weight updates within the BNPS function set. Convergence of the learned weights was verified by comparing both models with known true values.

In addition, a multi-membrane logistic regression architecture was developed and evaluated on the Iris dataset using 21 membranes, where a single controller membrane provides weights to 20 sample membranes and aggregates their parallel gradient outputs.

Benchmarking results of the serial and GPU-parallel simulator indicate that the speed of computation improves significantly with an increasing number of membranes, achieving a maximum speedup of $1.94\times$ for the 21-membrane Iris configuration compared to $1.17\times$ and $1.01\times$ for single-membrane models. These results demonstrate the feasibility of encoding gradient-descent-based machine learning algorithms natively within the BNPS membrane computing framework and confirm the suitability of this parallel execution architecture as a scalable platform for bio-inspired machine learning.

KEYWORDS: Membrane Computing; Numerical P Systems; Boolean Conditions; Gradient Descent; GPU-Parallel Simulation.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
LIST OF TABLES	vi
LIST OF FIGURES	vii
ABBREVIATIONS	viii
NOTATION	ix
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Objectives	2
2 LITERATURE SURVEY	3
2.1 Membrane Computing and P Systems	3
2.2 Numerical P Systems and Their Variants	4
2.3 P Systems Applied to Machine Learning	4
2.4 Simulators and Tools for Numerical P Systems	5
2.5 Cloud Workload Prediction	5
3 Problem Description	7
3.1 Problem Statement	7
3.1.1 BNPS Simulator Non-Existence	7
3.1.2 Limited Multi-Membrane Execution and GPU Support . . .	8
3.1.3 Lack of Native Machine Learning Support in BNPS	8
3.1.4 BNPS Models for Cloud Resource Utilisation Prediction . .	8

3.2	Problem and Solution Summary	8
3.2.1	Development of the BNPS Simulator	8
3.2.2	Encoding and Validation of Machine Learning Algorithms	9
3.2.3	Prediction of Cloud Resource Utilisation Using BNPS	9
4	Solution	10
4.1	Formal Definition of the BNPS	10
4.2	BNPS Simulator	11
4.2.1	Input Format	11
4.2.2	Serial Execution	11
4.2.3	GPU-Parallel Execution	11
4.3	Linear Regression in BNPS	12
4.4	Logistic Regression in BNPS	12
4.4.1	Sigmoid Approximation	12
4.4.2	Pipeline Architecture	13
4.5	Multi-Membrane Logistic Regression and the Iris Dataset	13
4.5.1	Architecture	13
4.5.2	Dataset Encoding	14
5	Results and Analysis	15
5.1	Linear Regression Convergence	15
5.2	Logistic Regression Convergence	16
5.3	Multi-Membrane Logistic Regression on the Iris Dataset	17
5.4	Serial vs GPU-Parallel Execution Performance	18
6	Summary and Future Scope	20
6.1	Summary	20
6.2	Future Scope	20
	REFERENCES	23

LIST OF TABLES

5.1	Serial vs GPU-Parallel execution time comparison across all three configurations (300 steps each).	18
-----	--	----

LIST OF FIGURES

5.1	Linear Regression — Weight convergence over 300 simulation steps. Dashed lines indicate the true target values ($w_1 = 1, w_2 = 2, b = 1$).	15
5.2	Logistic Regression — Weight convergence over 300 simulation steps. Dashed lines indicate the true target values ($w_1 = 2, w_2 = 2, b = 0$).	17
5.3	Multi-Membrane Logistic Regression on the Iris dataset — Weight convergence over 300 simulation steps across 21 membranes.	17
5.4	Serial vs GPU-Parallel execution time across all three BNPS configurations (300 steps each).	19

ABBREVIATIONS

BNPS	Numerical P Systems with Boolean Conditions
NPS	Numerical P Systems
MC	Membrane Computing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
ML	Machine Learning
MSE	Mean Squared Error
ENPS	Enzymatic Numerical P Systems
SNUPS	Sequential Numerical P System Simulator
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
SLA	Service Level Agreement
RPN	Reverse Polish Notation
SNP	Spiking Neural P Systems
NSN	Numerical Spiking Neural

NOTATION

Π	BNPS tuple representation
m	Number of membranes
H	Set of membrane identifiers
μ	Membrane enclosure structure
Var_i	Set of real-valued variables in membrane i
Pr	Set of all programs in the system
F	Arithmetic production function
C	Boolean condition
$c_j v_j$	Repartition coefficient and target variable
$\wedge$	Logical AND operator
$\vee$	Logical OR operator
$\neg$	Logical NOT operator
$\oplus$	Logical XOR operator
$\hat{y}$	Predicted output value
w_1, w_2	Model weights
b	Bias term
z	Linear combination of inputs
$\sigma(z)$	Sigmoid activation function
lr	Learning rate
∇	Gradient operator

CHAPTER 1

INTRODUCTION

1.1 Introduction

Biological cells perform extraordinarily complex computations in parallel across millions of membrane-separated compartments, inspiring researchers to ask whether the same organisational principles could be applied to solve computational problems [1]. From biological neural networks to evolutionary algorithms, there is an overwhelming amount of evidence to support this approach across the many areas of computing. For example, researchers are still developing models of how the cell works to use as a computational model through Membrane Computing (MC). Just as in a cell, when chemical reactions occur in parallel in many different compartments of a cell, separated by membranes, we can define a computational model in which each of the multiple computational compartments, having many parallel instances, work together through the use of controlled interactions defined by the membranes that separate them. Therefore, a general computational model for all of the aforementioned can be defined by the P system [2].

MC is an exciting and applicable area of research today. With machine learning being one of the most computationally intensive areas of computing and requiring various numerical calculations to train the model, run regressions, and classify data, most of the computations associated with machine learning are completed using conventional software on conventional hardware [3]. The central focus of this project is the following question: can we define an MC model from which to encode machine learning algorithms directly into Membrane Computing using its inherent parallel nature? In particular, this research focuses on the use of Numerical P Systems with Boolean Conditions (BNPS).

1.2 Motivation

With the continued growth of Machine Learning (ML) applications, there is an increasing demand for efficient systems. Although GPUs and TPUs provide hardware-based parallelism, an alternative approach is to use parallelism directly into the computational model. The inherently iterative nature of ML algorithms such as Gradient Descent (GD), combined with the way Numerical P Systems (NPS) process real-valued variables through arithmetic production rules, makes them suitable for implementing ML algorithms. However, no partially supervised ML algorithm has yet been simulated within Boolean Numerical P Systems (BNPS), primarily due to the absence of a dedicated BNPS simulator. This project addresses that gap by developing BNPS simulators and implementing linear and logistic regression and developing BNPS simulator for work load prediction. The objective is to reduce computation time while maintaining the same level of prediction accuracy as existing methods.

1.3 Objectives

This project has three primary objectives:

- **Objective 1:** Design and development of a parallel simulator for BNPS.
- **Objective 2:** Implementation of basic machine learning algorithms using BNPS.
- **Objective 3:** Design and development of BNPS based machine learning algorithm for cloud workload prediction

CHAPTER 2

LITERATURE SURVEY

This chapter reviews the state-of-the-art research relevant to our project within five headings: the theoretical foundation of P systems and membrane computing; numerical P systems variants; application of P systems to machine learning; P systems simulation tools; and cloud workload prediction.

2.1 Membrane Computing and P Systems

Membrane computing has been suggested as a paradigm of natural computation and was first introduced by Gh. Păun as being inspired by the molecular processes that occur in biological cells [1]. The idea behind P systems is that computation occurs within a hierarchical structure of membranes; objects can evolve according to predefined rules and migrate between regions through diffusion or transport processes. Therefore, this model of computation has the benefit of providing an inherently parallel and concurrent manner of performing computations [2]. P systems have been expanded to include many variations, mainly influenced by the requirements of the particular computational model and the need to take advantage of more than one type of component [4].

Two primary types of P systems have been suggested in the literature. Tissue P systems model the evolution of intercellular communication and use rules based on symport and antiport concepts; evolutionary variants of tissue P systems can solve NP-complete computational problems in polynomial time [5]. Examples of the application of cell-like P systems in conjunction with Particle Swarm Optimization to clustering problems have also been documented [6].

2.2 Numerical P Systems and Their Variants

Numerical P Systems (NPS) were presented in conjunction with membrane computing and economics by Păun [7]. Whereas classical P systems work with a multiset of abstract symbols such as atoms and strings, NPS use concrete numerical values through arithmetic production functions and repartition protocols to redistribute numbers across membranes based on computed values [7, 8].

NPS with Thresholds allow an execution program to run when the numerical variable meets a threshold condition, and have been studied with respect to their equivalence with Petri nets [9]. Enzymatic NPS introduced enzyme variables that influence rule applicability, originally developed for robot control, and are Turing universal even with degree-1 polynomial production functions [10, 11]. NPS with Migrating Variables allow numerical variables to be transferred from one membrane to another, providing flexibility in the ability to transfer data [12]. NSN P systems were created as an extension of Spiking Neural P systems and allow production functions to be defined by continuous rather than discrete spike counts [13, 14]. NPS with Boolean Conditions (BNPS) allow for better conditional control over the execution of a program than traditional conditional statements through the use of logical operators — AND ($\wedge$), OR ($\vee$), NOT ($\neg$), and XOR ($\oplus$) — to compare the values of numerical variables [15].

2.3 P Systems Applied to Machine Learning

There are multiple studies that involve the application of P systems in the performance of various machine learning tasks. The use of P systems for parallel decision tree classification [16], as well as the implementation of a k-nearest neighbour classifier optimised by P systems [17], have all been studied. Fault diagnosis and time-series classification have also been studied using spiking neural P systems [18, 19].

When considering the current findings, it is important to note that there are no instances of the use of native numerical variables as the basis for computation in any of these implementations. The majority of implementations have utilised P system structures as control wrappers surrounding conventional classifiers and have encoded data

as symbolic multisets or spikes. Classification and regression algorithms are inherently numerical, as they update real-valued weights based on gradient computations iteratively. No other research has employed NPS or any numerically native variant of P systems to implement gradient-descent-based machine learning algorithms within a membrane computing environment. The present work addresses this gap directly by encoding both linear regression and logistic regression within the BNPS framework, where each training sample occupies dedicated membrane, and by validating their convergence to correct solutions.

2.4 Simulators and Tools for Numerical P Systems

There exist many simulators and tools for the simulation of different types of NPS. Some of the first sequential NPS simulators were as follows.

- **SNUPS** — one of the first and most widely used sequential simulation tools specifically targeting NPS [20].
- **PeP** — a high-level sequential simulation tool written in Python specifically to simulate P systems [21].
- **GPUPeP** — a simulation tool written using CUDA to simulate P systems in a parallel computing environment, providing substantially reduced execution time compared with sequential execution [22].
- **Multi-ENPS** — an ENPS simulation tool that includes utilities for automatically transforming existing files into ENPS simulations [23].

No existing tools fully simulate BNPS using the Boolean conditions that BNPS requires. Furthermore, none support multi-membrane BNPS configurations where variables communicate across membrane boundaries during gradient-based learning. The present work addresses this by developing a simulator that supports both serial and GPU-parallel execution for single-membrane and multi-membrane BNPS models.

2.5 Cloud Workload Prediction

The cloud environment processes large volumes of dynamic workloads, creating an ongoing challenge to predict future resource requirements accurately. Failure to accu-

rately predict workloads may result in over- or under-provisioning of resources; over-provisioning will not only waste resources but also increase costs, while under-provisioning will diminish service quality when performance degradation occurs or SLA violations happen [24]. The task of workload prediction in the cloud is primarily a time series problem. CNN-LSTM hybrid models are particularly effective in this type of task due to their ability to extract complex features and model long-term temporal dependencies [25]. More advanced architectures such as TFEGRU and MSCNet have been proposed to better handle the complexity of cloud workloads, capturing patterns across both frequency and temporal domains to improve long-term prediction accuracy [26, 27].

To date there has been significant advancement in the field of cloud workload prediction; however, all previous approaches depend on traditional computing methods. No studies have used the encoding of cloud workload prediction within a membrane computing model. This study proposes to develop an alternative model by utilising the natural characteristics of BNPS in processing real-valued numerical variables through arithmetic production functions in a highly parallel manner.

CHAPTER 3

Problem Description

Cloud computing services are composed of constantly changing workloads that require different levels of resources in real-time based on user demands. It is very important to accurately predict future workloads, as inaccurate prediction may lead to wasteful resource usage (over-provisioning) or insufficient resource allocation resulting in reduced performance levels. Therefore, predicting future workloads is a critical task of resource management in cloud computing environments.

Currently, many workload prediction models are based on machine learning or deep learning approaches. Most of these models rely on conventional computing architectures for workload prediction. However, membrane computing has not yet been widely explored for workload prediction in cloud environments. In particular, very limited research has investigated the use of Numerical P Systems with Boolean Conditions (BNPS) for cloud workload prediction.

3.1 Problem Statement

Although advancements have been made in membrane computing and workload prediction, several limitations still remain.

3.1.1 BNPS Simulator Non-Existence

Existing simulators such as SNUPS, PeP, GPUPeP, and Multi-ENPS support different classes of P systems; however, they do not fully support the execution rules required for BNPS systems. In addition, the use of logical operators for forming Boolean conditions (AND, OR, NOT, and XOR) with real-valued variables is only partially supported, limiting practical experimentation using BNPS models.

3.1.2 Limited Multi-Membrane Execution and GPU Support

Some simulators provide support either for multi-membrane execution or GPU-based computation. However, no existing simulator supports GPU-parallel execution of multi-membrane BNPS simulations with inter-membrane communication. This limitation restricts scalability and efficient experimentation for large BNPS systems.

3.1.3 Lack of Native Machine Learning Support in BNPS

Previous applications of membrane computing in machine learning mainly rely on symbolic or spiking representations. The direct implementation of gradient descent using real-valued BNPS variables has received limited research attention, and numerical learning-based training methods are largely absent in existing BNPS implementations.

3.1.4 BNPS Models for Cloud Resource Utilisation Prediction

Traditional machine learning approaches are commonly used for workload prediction in cloud computing environments. BNPS-based models have received very little attention for predicting cloud resource utilisation, highlighting a clear research gap in this area.

3.2 Problem and Solution Summary

This project addresses the identified challenges through three main stages.

3.2.1 Development of the BNPS Simulator

A BNPS simulator supporting GPU-parallel execution for single and multi-membrane configurations will be developed. This simulator shall be base of the remaining project.

3.2.2 Encoding and Validation of Machine Learning Algorithms

Linear regression and logistic regression models will be implemented directly within the BNPS framework using arithmetic production rules. Convergence behaviour will be validated using synthetic datasets as well as the Iris dataset.

3.2.3 Prediction of Cloud Resource Utilisation Using BNPS

After validation, the BNPS framework will be extended to process time-series resource usage data for predicting cloud resource utilisation in cloud computing environments. This approach is anticipated to achieve substantial computational acceleration in cloud workload prediction while preserving the prediction accuracy at levels comparable to traditional implementations.

CHAPTER 4

Solution

This chapter explains the structure of the BNPS simulator plus an overview of the design methodology used to create the simulator, as well as encode various machine learning algorithms into the membrane computing model. The design consists of three stages, which include the simulator's development, encoding linear and logistic regression models from synthetic datasets and extending the two models to a multi-membrane architecture using real-world datasets.

4.1 Formal Definition of the BNPS

A Numerical P System with Boolean Conditions is represented as the Tuple:

$$\Pi = (m, H, \mu, (Var_1, Var_1(0)), \dots, (Var_m, Var_m(0)), Pr)$$

where m is the number of membranes, H is the set of membrane identifiers, μ represents the enclosure structure of the membranes, Var_i represents the entire set of real-valued variables of membrane i with $Var_i(0)$ representing the initial value of those variables, and Pr is the collective set of all programs controlling the dynamics of the system.

Each program has the form:

$$F(\text{variables}) \mid C(\text{variables}) \rightarrow c_1|v_1 + c_2|v_2 + \dots + c_n|v_n$$

where F represents an arithmetic production function, C represents a Boolean condition created using the AND ($\wedge$), OR ($\vee$), NOT ($\neg$), and XOR ($\oplus$) operators based on comparisons between real-valued variables, and $c_j|v_j$ represents the repartition coefficient that determines how much of the produced value should go to each of the target variables.

Execution is performed in three phases per step:

1. All program Boolean conditions are evaluated using variable snapshots from the previous execution step.
2. Active programs perform their production functions simultaneously, with consumed variables reset to zero.
3. The produced values are distributed to the appropriate target variables based on repartition coefficients, with contributions from multiple programs summed together.

4.2 BNPS Simulator

4.2.1 Input Format

The simulator accepts `.pep` input files that describe the membrane structure, variables, initial values, and production rules. A conditional program is written as:

```
production_function | boolean_condition -> repartition
```

An unconditional program is written as:

```
production_function -> repartition
```

4.2.2 Serial Execution

The serial simulator (`bnps.py`) reads the `.pep` input file and simulates the three phases for each step sequentially using Python. Predicates are evaluated using variable snapshots from the previous step. Once all active programs have computed their production values with consumed variables set to zero, the target variables are updated simultaneously.

4.2.3 GPU-Parallel Execution

The parallel simulator uses `bnps.py` to read the `.pep` file and generates an intermediate file (`input.txt`), which is passed to the CUDA kernel (`bnps.cu`) for execution.

GPU threads evaluate all programs across all membranes simultaneously within each execution step while preserving the snapshot semantics. For any valid input configuration, both serial and parallel modes produce identical output.

4.3 Linear Regression in BNPS

The model to be trained is:

$$\hat{y} = w_1x_1 + w_2x_2 + b \quad (4.1)$$

This model is trained using batch gradient descent while the loss function is the Mean Squared Error (MSE). The gradient descent algorithm is divided into a seven-stage pipeline within a single BNPS membrane.

- **Stage 1:** The master weights are distributed to per-sample and temporary slots using the multiplication-distribution method.
- **Stage 2:** A fresh learning rate token is injected for every simulation step.
- **Stage 3:** Predictions are computed for all samples simultaneously in one step.
- **Stage 4:** Each prediction is duplicated into three different gradient slots.
- **Stage 5:** Gradients are calculated for every sample, skipping the case when the prediction is not equal to the target.
- **Stage 6:** All gradients calculated for every sample are aggregated together.
- **Stage 7:** Weights are updated using the aggregated gradients. When the total gradient reaches zero, Boolean conditions halt all weight updates automatically, embedding convergence detection directly into the execution semantics.

4.4 Logistic Regression in BNPS

4.4.1 Sigmoid Approximation

Since the BNPS production function set does not include the exponential function, the standard sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is replaced with the piecewise-linear approximation:

$$\sigma(z) \approx 0.5 + 0.25z$$

Using this approximation, the gradient of the loss L with respect to a weight w becomes:

$$\frac{\partial L}{\partial w} = (0.25z + 0.5 - y) \cdot x$$

4.4.2 Pipeline Architecture

The pipeline is constructed in a similar staged manner as the linear regression model. The primary difference is that the linear combination $z_i = w_1x_{1,i} + w_2x_{2,i} + b$ is computed first, and gradients are then derived from the linearised sigmoid. Boolean conditions for $y = 1$ samples fire when $z_i \neq +2$, and for $y = 0$ samples fire when $z_i \neq -2$, since under the linearised model convergence occurs at $z = +2$ for positive samples and $z = -2$ for negative samples.

4.5 Multi-Membrane Logistic Regression and the Iris Dataset

4.5.1 Architecture

The multi-membrane architecture assigns dedicated roles to each membrane. Membrane 1 acts as the controller, holding the master weights, broadcasting weight copies to all sample membranes, aggregating gradients received from sample membranes, and updating the master weights. Membranes 2 through 21 are sample membranes, with one membrane created per training sample. Each sample membrane receives a copy of the controller weights, computes z_i and the per-sample gradients, and returns the gradients to the controller.

Weight data flows downward from the controller to all sample membranes through the repartition mechanism, and gradient data flows upward from sample membranes

back to the controller through the same mechanism. All 20 sample membranes compute their gradients simultaneously within a single simulation step.

4.5.2 Dataset Encoding

Twenty samples from the Iris dataset are used — 10 from the Setosa class ($y = 0$) and 10 from the Non-Setosa class ($y = 1$) — with all four features (sepal length, sepal width, petal length, petal width) standardised using z-score normalisation. Here, a real-life dataset is used to evaluate the practical applicability of the proposed model.

CHAPTER 5

Results and Analysis

The results from three sets of experiments are presented in this chapter: the convergence validation of linear regression implemented in a single-membrane BNPS, the convergence validation of logistic regression implemented in a single-membrane BNPS, and weight convergence of a multi-membrane logistic regression trained on the Iris dataset. Execution times for serial and GPU-parallel execution are compared across all three configurations to validate the parallel execution capability of the simulator.

5.1 Linear Regression Convergence

The single-membrane linear regression model was run serially for a period of 300 simulation steps. In selecting the six synthetic samples used to create the dataset, the true values of w_1 , w_2 , and b were chosen to be 1, 2, and 1 respectively. Using batch gradient descent with a starting value of zero for each weight, the weights were updated at each step with a learning rate of 0.015 using the Mean Squared Error loss function.

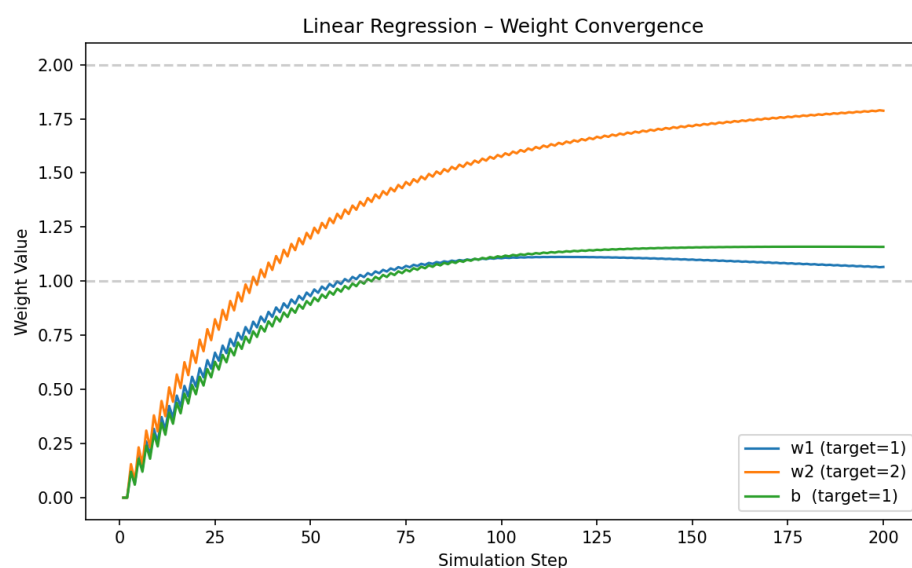


Figure 5.1: Linear Regression — Weight convergence over 300 simulation steps. Dashed lines indicate the true target values ($w_1 = 1$, $w_2 = 2$, $b = 1$).

The three parameters of the single-membrane linear regression model converged towards their target values in a steady-state fashion. When the simulation was completed at the 300th step, w_1 converged to approximately 1.07, b converged to approximately 1.16, and w_2 converged to approximately 1.79, all reaching their true values of 1, 1, and 2 respectively. The pattern of the parameters during the first few steps supports the four-step latency in the BNPS snapshot semantics that causes weight updates to lag behind gradient computations by several simulation steps. This type of behaviour does not mean less stability. The Boolean mechanism for detecting convergence embedded in Stage 7 behaves in such a way that it is able to prevent weight updates to any sample whose gradient goes to zero, and therefore it shows that the conditional halting logic functions as intended.

5.2 Logistic Regression Convergence

A single-membrane logistic regression model was simulated for 300 iterations using four synthetic binary classification datasets with known true values of $w_1 = 2$, $w_2 = 2$, and $b = 0$. The standard sigmoid function was approximated by the piecewise-linear approximation $\sigma(z) \approx 0.5 + 0.25z$, and a learning rate of 0.5 was used during the simulation. The simulation showed that positive samples converged at $z = +2$, whereas negative samples converged at $z = -2$.

The results indicated that w_1 and w_2 reached to their target values of 2.0 after approximately 300 iterations, while b remained at a value of 0.0 during the simulation. This finding corresponds with the symmetric structure of the dataset, which results an aggregate bias gradient of zero. The Boolean guard protecting updates to w_1 and w_2 correctly ensures that these parameters are not changed when their gradients are zero, while the unconditional update of the bias correctly maintains b when the bias gradient is zero. The bias weight trace yielded the same value as the target dashed line throughout the entire simulation, thus demonstrating appropriate treatment of this edge case.

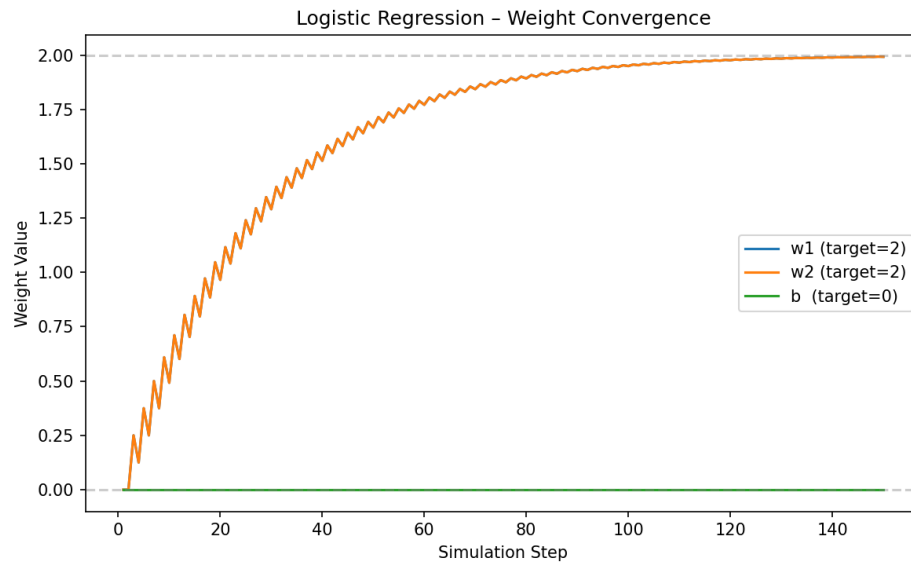


Figure 5.2: Logistic Regression — Weight convergence over 300 simulation steps. Dashed lines indicate the true target values ($w_1 = 2$, $w_2 = 2$, $b = 0$).

5.3 Multi-Membrane Logistic Regression on the Iris Dataset

The multi-membrane architecture was evaluated using 20 samples from the Iris dataset — 10 from the Setosa class ($y = 0$) and 10 from the Non-Setosa class ($y = 1$) — where each of the four features are standardised using z-score normalisation. Membrane 1 is the controller that contains the master weights w_1 , w_2 , w_3 , w_4 , and b , while Membranes 2 through 21 each host a single training sample. The model was run for 300 steps with a learning rate of 0.1.

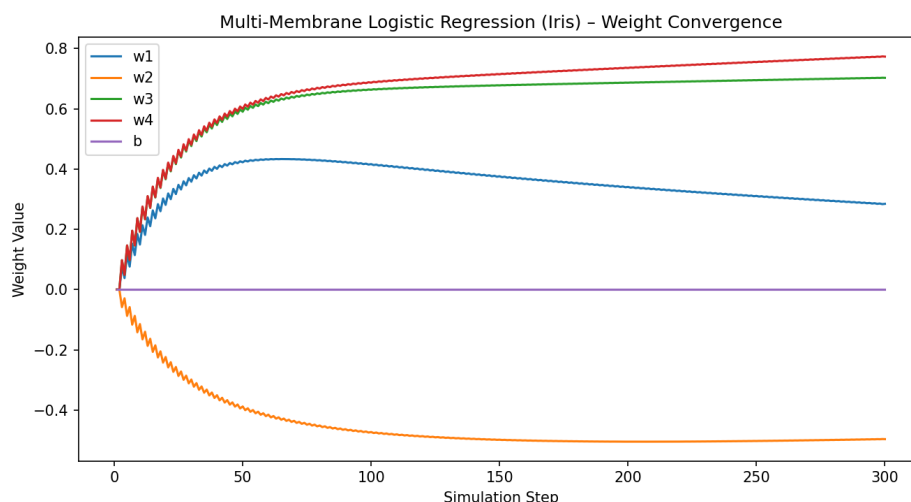


Figure 5.3: Multi-Membrane Logistic Regression on the Iris dataset — Weight convergence over 300 simulation steps across 21 membranes.

Each of the parameters evolved smoothly from zero to a stable value. w_3 (petal length) converged to approximately 0.70, while w_4 (petal width) converged to approximately 0.78, demonstrating the known ability of petal length and petal width features in the Iris dataset to discriminate classes from one another. For w_1 (sepal length), the weight peaked at around 0.43 near step 90 and then dropped to around 0.29 after approximately 300 steps. This behaviour is consistent with the fact that the Setosa versus Non-Setosa classification task relies more on petal length and width than sepal length for determining class membership. In contrast, w_2 (sepal width) has a strong negative weight of around -0.48 , reflecting that Setosa samples typically have greater sepal widths compared to other classes in the Iris dataset. Finally, b remains near zero throughout the process. The consistent convergence patterns observed across all 21 membranes verify that the controller's weight broadcast and gradient aggregation mechanism functions correctly within the multi-membrane BNPS framework.

5.4 Serial vs GPU-Parallel Execution Performance

The computational advantage of GPU-parallel execution was evaluated by running the simulator in both serial and parallel modes for each of the three configurations described above over 300 steps. The parallel mode of execution creates an intermediate `input.txt` file that is passed to the CUDA kernel (`bnps_fixed.cu`) running on an NVIDIA T4 GPU. The results are given in Table 5.1.

Table 5.1: Serial vs GPU-Parallel execution time comparison across all three configurations (300 steps each).

Configuration	Serial (ms)	GPU Parallel (ms)	Speedup
Linear Regression (1 membrane)	254.32	217.17	$1.17\times$
Logistic Regression (1 membrane)	211.64	209.58	$1.01\times$
Iris Logistic Regression (21 membranes)	531.54	274.01	$1.94\times$

The data demonstrate a distinct pattern: there is little to no speed advantage of utilising GPU parallelism for single-membrane architectures, but there are significant advantages found when utilising GPU parallelism for configurations involving more than one membrane. The single-membrane linear regression model shows a speedup of $1.17\times$, while the single-membrane logistic regression model shows very less speedup of $1.01\times$.

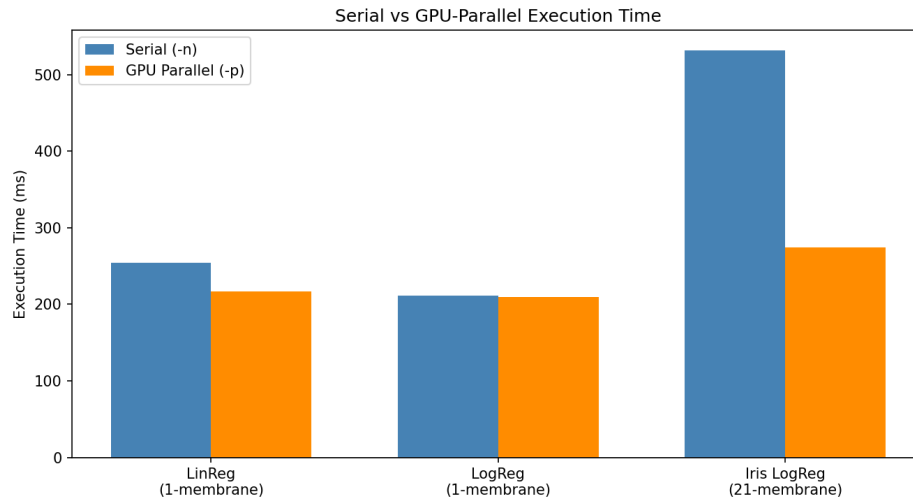


Figure 5.4: Serial vs GPU-Parallel execution time across all three BNPS configurations (300 steps each).

This was predicted as there is no inter-membrane parallelism in single-membrane models; the CPU overhead for the creation of `input.txt`, compilation of the CUDA kernel, and launching of GPU threads decreases any speed benefits came from the parallel execution.

On the contrary, the 21-membrane Iris logistic regression configuration achieved a speedup of $1.94\times$, reducing the execution time from 531.54 ms to 274.01 ms. This demonstrates that as the number of membranes increases, the GPU is able to compute all membrane programs concurrently within a single simulation step, allowing for better performance improvements and validating the design principle of the parallel simulator. The confirmation that identical final variable values were produced by both serial and parallel modes across all configurations confirms that the GPU execution correctly maintains the snapshot semantics of the BNPS.

CHAPTER 6

Summary and Future Scope

6.1 Summary

The overall goal of this project was to develop a framework for encoding machine learning algorithms onto the membrane computing paradigm using a BNPS. A simulator was developed which provides two execution modes: one using a single membrane and another using multiple membranes in parallel on the GPU, with both modes completing the full three-phase execution semantics of BNPS.

Linear regression and logistic regression were natively encoded into the BNPS framework using an iterative pipeline architecture. The execution of both models was validated by confirming that the predicted weight values converged to their known true weight values.

To demonstrate the effectiveness of the parallel multi-membrane BNPS architecture, the multi-membrane version of logistic regression was evaluated on the Iris dataset using 21 membranes. Benchmarking confirmed that as additional membranes were added, there was a corresponding speedup when executing on the GPU, achieving a speedup of $1.94\times$ for the 21-membrane configuration.

6.2 Future Scope

In order to complete the next stage of this research, the validated simulator and multi-membrane architecture developed in this project will serve as the basis for encoding a BNPS-based model for predicting cloud workloads. The framework will be extended to process resource utilisation data from cloud service providers to forecast anticipated CPU, memory, and network demands. It is anticipated that this model will be able to take advantage of the inherent parallelism of the multi-membrane BNPS architecture to

achieve prediction accuracy comparable to traditional deep learning approaches, while operating entirely within the membrane computing paradigm.